LESSON 06 · CHEAT SHEET

Errors & Debugging

READING A TRACEBACK

```
Traceback (most recent call last):
  File "my_work/ex1.py", line 4, in <module>
    print(employees[i])          ← WHERE (your line)
IndexError: list index out of range
↑ WHAT: type and message. Read this first.
```

COMMON EXCEPTION TYPES

`ValueError`	right type, bad content: `float("n/a")`
`TypeError`	wrong type: `float(None)`, `"5" + 3`
`KeyError`	dict key missing: `row["Salary"]`
`IndexError`	list position missing: `items[9]`
`ZeroDivisionError`	`x / 0`
`FileNotFoundError`	wrong path or folder
`NameError`	typo in a variable name
`AttributeError`	`"abc".uper()`
`SyntaxError`	Python can't even read the file

THE TRY FAMILY

```
try:
    salary = float(text)      # the risky line only
except ValueError:
    print("not a number")     # if it failed
else:
    total += salary           # if it worked
finally:
    checked += 1              # always
```

CATCHING VARIATIONS

```
except ValueError as e:       # keep the message
    print("Problem:", e)
except (ValueError, TypeError):   # several
except ZeroDivisionError:         # one block each
    ...
except ValueError:
    ...
type(e).__name__          'ValueError'
```

Never a bare `except:`. It's `On Error Resume Next` with extra steps.

RAISING YOUR OWN

```
def check_grade(grade):
    if not isinstance(grade, int):
        raise TypeError(f"not a number: {grade!r}")
    if not 1 <= grade <= 6:
        raise ValueError(f"not 1-6: {grade}")
    return grade
```

A REUSABLE SAFETY NET

```
def safe_float(text, default=None):
    try:
        return float(text)
    except (ValueError, TypeError):
        return default

safe_float("n/a")         None
safe_float("", 0.0)       0.0
```

DEBUGGING CHECKLIST

1. Read the last line of the traceback
2. Go to the line number. What are the values *right there*?
3. `print(repr(x))` or `print(type(x))` just before it
4. Make the input smaller until it works, then grow it
5. Wrong number, no crash? Print inside the loop and watch it change

VS CODE DEBUGGER

click left of a line number	breakpoint (red dot)
`F5`	run until the breakpoint
`F10`	step over (next line)
`F11`	step into a function
`F5` again	continue
Variables panel	every value, live

GOTCHAS

- Ten lines in one `try`: you won't know which one failed
- `except Exception: pass` hides bugs forever
- Catching an error is not fixing it. Log what you skipped
- Zero hours is a valid float. Not every wrong value raises